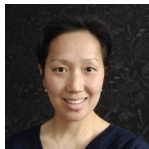


# 60016 - Linear & mixed-integer linear optimisation of trained ReLU neural networks

Ruth Misener

# Collaborators



Elena  
Botoeva  
Kent



Panagiotis  
Kouvaros  
Imperial



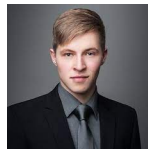
Jan  
Kronqvist  
KTH



Alessio  
Lomuscio  
Imperial



Ruth  
Misener  
Imperial



Alex  
Thebelt  
Imperial

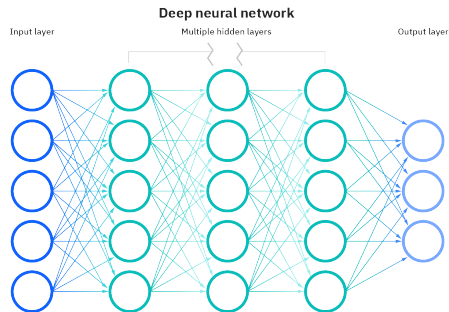


Calvin  
Tsay  
Imperial

**Papers** Lomuscio and Maganti [2017] • Botoeva et al. [2020] • Kronqvist et al. [2021] • Tsay et al. [2021]

First paper: Lalit Maganti's UG research project in the Department of Computing!

# Neural networks [In case anyone has been hiding under a rock ...]



<https://www.ibm.com/uk-en/cloud/learn/neural-networks>

- Input layer, hidden layer(s), output layers,
- Picture is fully dense feed-forward
- Other architectures: Convolutional neural networks (CNN), recurrent neural networks (RNN),
- Train with data & optimisation (COMP70007),
- Activation function at each node (we focus on ReLU, rectified linear activation function)

# Adversarial example

[Goodfellow et al., 2015]

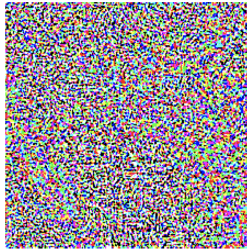


$x$

“panda”

57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

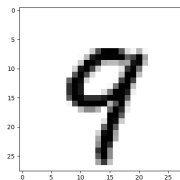
$=$



$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$   
“gibbon”

99.3 % confidence

# Optimisation problems over trained neural networks



Given .....

Trained NN

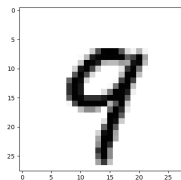
Image  $\bar{x}$

Label  $j = 9$

Adversary?  $k = 4$

- **Verification** [*Feasibility*] Is there an adversary labeled  $k$  within a given perturbation  $\ell_1$  or  $\ell_\infty$ ?
- **Optimal adversary** What image within perturbation  $\ell_1$  or  $\ell_\infty$  maximises the prediction difference?
- **Minimally distorted adversary** [Croce and Hein, 2020] Smallest perturbation  $\ell_1$  or  $\ell_\infty$  over which the NN predicts label  $k$ ?
- **Lossless compression** [Serra et al., 2020] Can I safely remove NN nodes or layers?

# Optimisation problems over trained neural networks



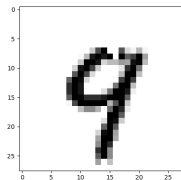
Given .....

Trained NN

Image  $\bar{x}$

Label  $j = 9$

Adversary?  $k = 4$

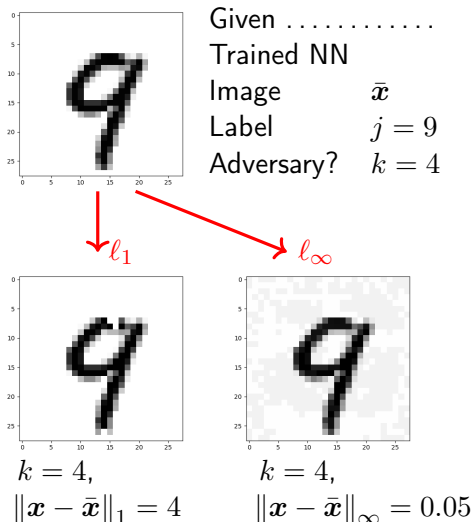


$k = 4,$

$$\|x - \bar{x}\|_1 = 4$$

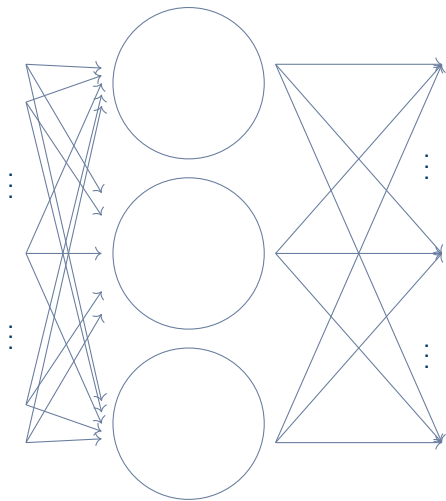
- **Verification** [*Feasibility*] Is there an adversary labeled  $k$  within a given perturbation  $\ell_1$  or  $\ell_\infty$ ?
- **Optimal adversary** What image within perturbation  $\ell_1$  or  $\ell_\infty$  maximises the prediction difference?
- **Minimally distorted adversary** [Croce and Hein, 2020] Smallest perturbation  $\ell_1$  or  $\ell_\infty$  over which the NN predicts label  $k$ ?
- **Lossless compression** [Serra et al., 2020] Can I safely remove NN nodes or layers?

# Optimisation problems over trained neural networks



- **Verification** [*Feasibility*] Is there an adversary labeled  $k$  within a given perturbation  $\ell_1$  or  $\ell_\infty$ ?
- **Optimal adversary** What image within perturbation  $\ell_1$  or  $\ell_\infty$  maximises the prediction difference?
- **Minimally distorted adversary** [Croce and Hein, 2020] Smallest perturbation  $\ell_1$  or  $\ell_\infty$  over which the NN predicts label  $k$ ?
- **Lossless compression** [Serra et al., 2020] Can I safely remove NN nodes or layers?

## Decision-making over a learned ReLU neural network



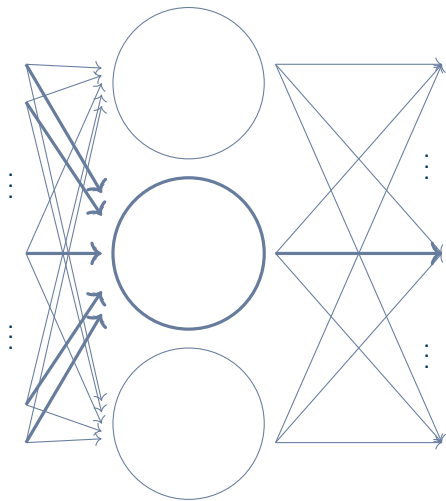
$$x^\ell = \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^\ell \geq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^\ell \leq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$



## Decision-making over a learned ReLU neural network

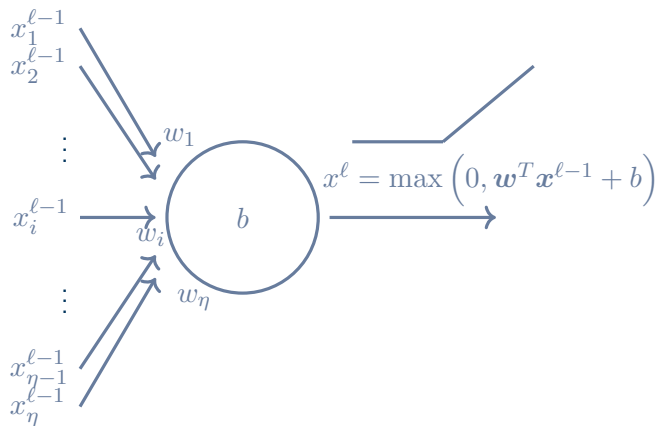


$$x^\ell = \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^\ell \geq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^\ell \leq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

## Decision-making over a learned ReLU neural network

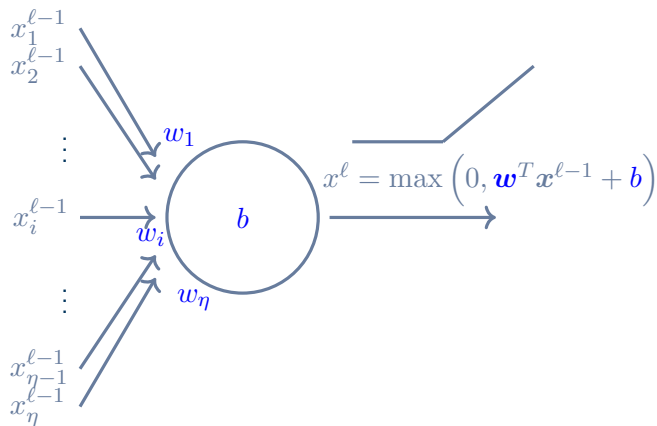


$$x^{\ell} = \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^{\ell} \geq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^{\ell} \leq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

## Decision-making over a learned ReLU neural network

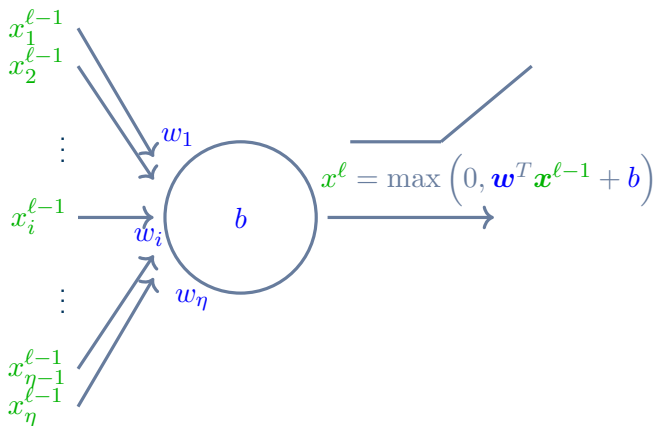


$$x^{\ell} = \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^{\ell} \geq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

$$x^{\ell} \leq \max \left( 0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \right)$$

## Decision-making over a learned ReLU neural network



$$\mathbf{x}^\ell = \max\left(0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b)\right)$$

$$\mathbf{x}^\ell \geq \max\left(0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b)\right)$$

$$\mathbf{x}^\ell \leq \max\left(0, (\mathbf{w}^T \mathbf{x}^{\ell-1} + b)\right)$$

## Verification [sign check only] & Optimal Adversary

$$\begin{aligned} \max \quad & f_k(\mathbf{x}^L) - f_j(\mathbf{x}^L) \\ \text{s.t.} \quad & \mathbf{x}_i^\ell = \max \left( 0, \left( \left( \mathbf{w}_i^{\ell-1} \right)^T \mathbf{x}^{\ell-1} + b \right) \right) \quad \forall \ell \in \{1, \dots, L\} = \text{Layer}, i \in \text{Node}^\ell \\ & \mathbf{x} \in \mathcal{X} \end{aligned}$$

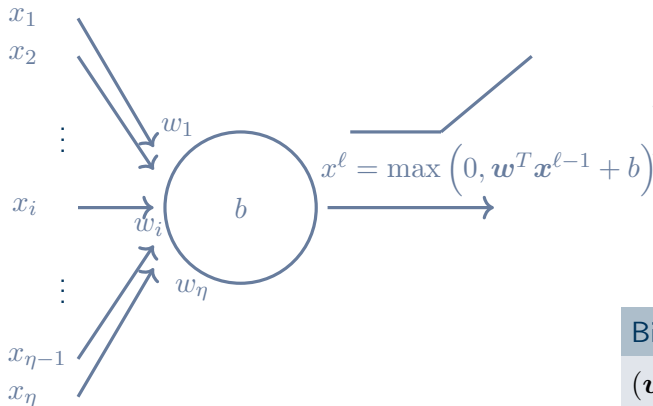
Here,  $f_k$  and  $f_j$  correspond to the  $k$ - and  $j$ -th elements of the neural network output layer  $L$ , respectively.  $\mathcal{X}$  defines the domain of perturbations.

## Expanding ...

$$\begin{aligned} \max \quad & f_k(\mathbf{x}^L) - f_j(\mathbf{x}^L) \\ \text{s.t.} \quad & \mathbf{x}_i^\ell \leq \max \left( 0, \left( \left( \mathbf{w}_i^{\ell-1} \right)^T \mathbf{x}^{\ell-1} + b \right) \right) \quad \forall \ell \in \{1, \dots, L\} = \text{Layer}, i \in \text{Node}^\ell \\ & \mathbf{x}_i^\ell \geq \max \left( 0, \left( \left( \mathbf{w}_i^{\ell-1} \right)^T \mathbf{x}^{\ell-1} + b \right) \right) \quad \forall \ell \in \{1, \dots, L\} = \text{Layer}, i \in \text{Node}^\ell \\ & \mathbf{x} \in \mathcal{X} \end{aligned}$$

# Big-M formulation of a learned ReLU neural network

Lomuscio and Maganti [2017], Fischetti and Jo [2018]



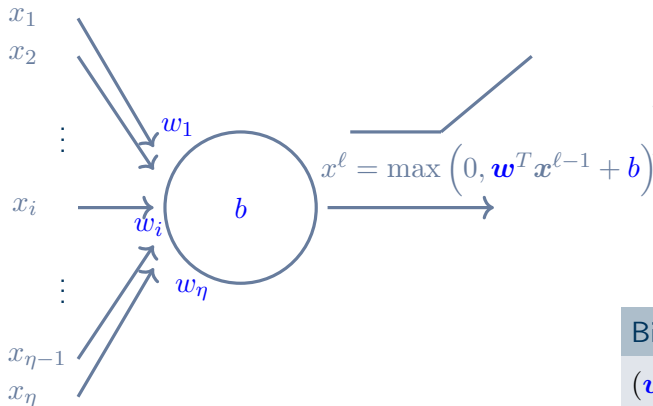
$$\begin{aligned}x^{\ell} &\geq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \\x^{\ell} &\leq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) - (1 - \sigma) LB^0 \\0 &\leq x^{\ell} \leq \sigma UB^0 \\ \sigma &\in \{0, 1\}\end{aligned}$$

Big-M coefficients  $LB^0, UB^0 \in \mathbb{R}$

$$(\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \in [LB^0, UB^0]$$

# Big-M formulation of a learned ReLU neural network

Lomuscio and Maganti [2017], Fischetti and Jo [2018]



$$\begin{aligned}x^{\ell} &\geq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \\x^{\ell} &\leq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) - (1 - \sigma) LB^0 \\0 &\leq x^{\ell} \leq \sigma UB^0 \\ \sigma &\in \{0, 1\}\end{aligned}$$

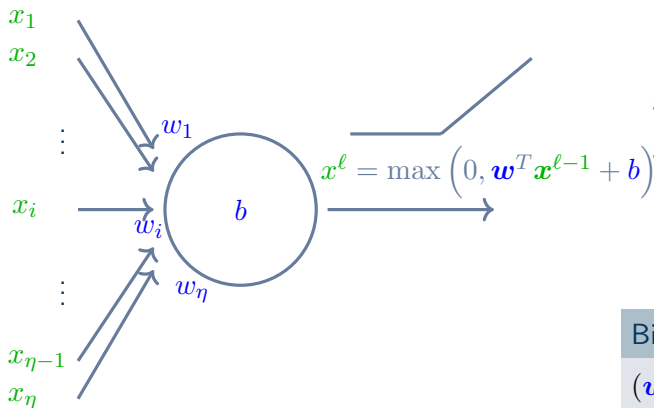
Big-M coefficients  $LB^0, UB^0 \in \mathbb{R}$

$$(\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \in [LB^0, UB^0]$$



# Big-M formulation of a learned ReLU neural network

Lomuscio and Maganti [2017], Fischetti and Jo [2018]



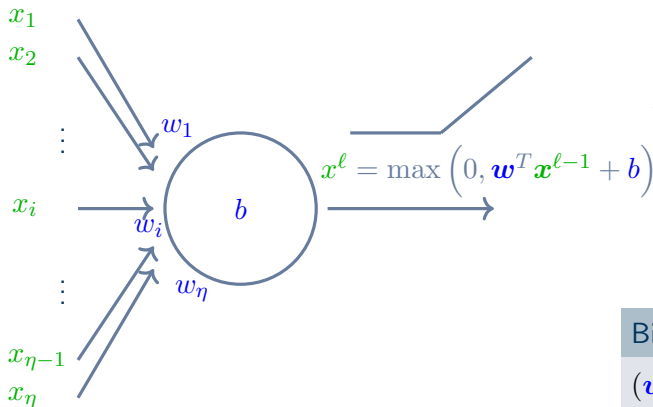
$$\begin{aligned}x^{\ell} &\geq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \\x^{\ell} &\leq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) - (1 - \sigma) LB^0 \\0 &\leq x^{\ell} \leq \sigma UB^0 \\ \sigma &\in \{0, 1\}\end{aligned}$$

Big-M coefficients  $LB^0, UB^0 \in \mathbb{R}$

$$(\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \in [LB^0, UB^0]$$

# Big-M formulation of a learned ReLU neural network

Lomuscio and Maganti [2017], Fischetti and Jo [2018]



$$\begin{aligned}x^{\ell} &\geq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \\x^{\ell} &\leq (\mathbf{w}^T \mathbf{x}^{\ell-1} + b) - (1 - \sigma) LB^0 \\0 &\leq x^{\ell} \leq \sigma UB^0 \\ \sigma &\in \{0, 1\}\end{aligned}$$

Big-M coefficients  $LB^0, UB^0 \in \mathbb{R}$

$$(\mathbf{w}^T \mathbf{x}^{\ell-1} + b) \in [LB^0, UB^0]$$

## Example 1 [Anderson et al., 2020]

Consider a NN node with a ReLU activation function that has 2 inputs ( $x_1$  and  $x_2$ ) & one output ( $y$ ). The bounds on the inputs are  $x_1, x_2 \in [0, 1]$  and we have parameters  $w_1 = w_2 = 1$  and  $b = -1.5$ .

- How can we formulate this node using the big-M method?
- Show that the point  $(x_1, x_2, y, \sigma) = (1, 0, 0.25, 0.5)$  is feasible for the LP relaxation.
- Show that the inequality  $y \leq 0.5x_2$  is valid for the ReLU activation function but not satisfied by the point  $(x_1, x_2, y, \sigma) = (1, 0, 0.25, 0.5)$ .

Note: The linear programming relaxation of the big-M formulation may be tightened with inequalities. Whether we *want* to add these tightening inequalities depends on the application.

## Example 1 Answer

$$y \geq x_1 + x_2 - 1.5$$

$$y \leq x_1 + x_2 - 1.5 - (1 - \sigma)LB^0$$

$$y \leq \sigma UB^0$$

$$x_1, x_2 \in [0, 1], y \geq 0, \sigma \in \{0, 1\}$$

What should be the value of  $LB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = 0$  and  $y = 0$ , so set  $LB^0 = -1.5$ .

What should be the value of  $UB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = 1$  and  $y = 0.5$ , so set  $UB^0 = 0.5$ .

Substituting in, we see that  $(x_1, x_2, y, \sigma) = (1, 0, 0.25, 0.5)$  is feasible if the constraint  $\sigma \in \{0, 1\}$  becomes  $\sigma \in [0, 1]$ .

The inequality is certainly satisfied when  $y = 0$ :  $0 \leq 0.5x_2$  must be true because  $x_2 \geq 0$ .

When  $y = x_1 + x_2 - 1.5$ , the maximum value of  $y$  is 0.5. Substituting into the equations, we can see that the LP relaxation point is not feasible in the ReLU node.

## Example 2

Consider a NN node with a ReLU activation function that has 3 inputs ( $x_1$ ,  $x_2$ , and  $x_3$ ). The bounds on the inputs are  $x_1, x_2, x_3 \in [0, 1]$  and we have parameters  $w_1 = w_2 = w_3 = 1$  and  $b = -1.5$ . How can we formulate this node using the big-M method?

## Example 2 Answer

$$y \geq x_1 + x_2 + x_3 - 1.5$$

$$y \leq x_1 + x_2 + x_3 - 1.5 - (1 - \sigma)LB^0$$

$$y \leq \sigma UB^0$$

$$x_1, x_2, x_3 \in [0, 1], y \geq 0, \sigma \in \{0, 1\}$$

What should be the value of  $LB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = x_3 = 0$  and  $y = 0$ , so set  $LB^0 = -1.5$ .

What should be the value of  $UB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = x_3 = 1$  and  $y = 1.5$ , so set  $UB^0 = 1.5$ .

### Example 3 [Anderson et al., 2020]

Fix  $\gamma \in \mathbb{R}_{\geq 0}$  and even  $\eta \in \mathbb{N}$ . Take the affine activation function  $\text{ReLU}(x) = \max(0, \sum_{i=1}^{\eta} x_i)$ , the input domain  $[L, U] = [-\gamma, \gamma]^{\eta}$  and  $\hat{x} = \gamma \cdot (1, -1, \dots, 1, -1)$  as a scaled vector that alternates positive and negative. Check that  $(\hat{x}, \hat{y}, \hat{\sigma}) = (\hat{x}, \frac{1}{2}\gamma\eta, \frac{1}{2})$  is feasible for the LP relaxation of the big-M formulation.

## Example 3 Answer

$$y \geq \sum_{i=0}^{\eta} x_i$$

$$y \leq \sum_{i=0}^{\eta} x_i - (1 - \sigma)LB^0$$

$$y \leq \sigma UB^0$$

$$x_1, x_2, x_3 \in [0, 1], y \geq 0, \sigma \in \{0, 1\}$$

What should be the value of  $LB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = x_3 = -\gamma$  and  $y = 0$ , so set  $LB^0 = -\gamma\eta$ .

What should be the value of  $UB^0$ ? We need this constraint to be inactive when  $x_1 = x_2 = x_3 = \gamma$  and  $y = \gamma\eta$ , so set  $UB^0 = \gamma\eta$ .

The proposed point is indeed feasible.



## How can we make this better?

- **Improved branching heuristics.** Henriksen and Lomuscio [2021], Hashemi et al. [2021]
- **Improve linear programming relaxations for one node.** Zhang et al. [2018], Xu et al. [2020]
- **Improve linear programming relaxations for several nodes.** Müller et al. [2022]

## References I

- Ross Anderson, Joey Huchette, Will Ma, Christian Tjandraatmadja, and Juan Pablo Vielma. Strong mixed-integer programming formulations for trained neural networks. *Mathematical Programming*, pages 1–37, 2020.
- Elena Botoeva, Panagiotis Kouvaros, Jan Kronqvist, Alessio Lomuscio, and Ruth Misener. Efficient verification of ReLU-based neural networks via dependency analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3291–3299, 2020.
- Francesco Croce and Matthias Hein. Minimally distorted adversarial examples with a fast adaptive boundary attack. In *International Conference on Machine Learning*, pages 2196–2205. PMLR, 2020.
- Matteo Fischetti and Jason Jo. Deep neural networks and mixed integer linear optimization. *Constraints*, 23(3):296–309, 2018.

## References II

- Vahid Hashemi, Panagiotis Kouvaros, and Alessio Lomuscio. Osip: Tightened bound propagation for the verification of relu neural networks. In *International Conference on Software Engineering and Formal Methods*, pages 463–480. Springer, 2021.
- Patrick Henriksen and Alessio Lomuscio. Deepsplit: An efficient splitting method for neural network verification via indirect effect analysis. In *IJCAI*, pages 2549–2555, 2021.
- Jan Kronqvist, Ruth Misener, and Calvin Tsay. Between steps: Intermediate relaxations between big-M and convex hull formulations. *CPAIOR*, 2021.
- Alessio Lomuscio and Lalit Maganti. An approach to reachability analysis for feed-forward ReLU neural networks. *arXiv preprint arXiv:1706.07351*, 2017.
- Mark Niklas Müller, Gleb Makarchuk, Gagandeep Singh, Markus Püschel, and Martin Vechev. Prima: general and precise neural network certification via scalable convex hull approximations. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–33, 2022.

## References III

- Thiago Serra, Abhinav Kumar, and Srikumar Ramalingam. Lossless compression of deep neural networks. In *Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 417–430. Springer, 2020.
- Calvin Tsay, Jan Kronqvist, Alexander Thebelt, and Ruth Misener. Partition-based formulations for mixed-integer optimization of trained relu neural networks. *NeurIPS*, 2021.
- Kaidi Xu, Huan Zhang, Shiqi Wang, Yihan Wang, Suman Jana, Xue Lin, and Cho-Jui Hsieh. Fast and complete: Enabling complete neural network verification with rapid and massively parallel incomplete verifiers. In *International Conference on Learning Representations*, 2020.
- Huan Zhang, Tsui-Wei Weng, Pin-Yu Chen, Cho-Jui Hsieh, and Luca Daniel. Efficient neural network robustness certification with general activation functions. *Advances in neural information processing systems*, 31, 2018.